
UNIT 5: DATA AGGREGATION AND GROUP OPERATIONS

Structure	Page No.
5.0 Introduction	117
5.1 Objectives	117
5.2 Group By Mechanics	117
5.2.1 Iterating Over Groups	121
5.2.2 Selecting a Columnar Subset of a Group	123
5.3 Data Aggregation	125
5.3.1 Multiple Aggregate Functions in a Column	126
5.3.2 Column-wise Aggregation	127
5.4 Grouping with Dictionaries and Series	129
5.5 Grouping with Functions	132
5.6 Summary	134
5.7 Answers/Solutions	134

5.0 INTRODUCTION

In the last unit of this block, we have discussed about hierarchical indexing, merging, concatenating, and combining datasets. We also discussed some of the techniques for reshaping the data. This unit explores an innovative approach to creating groupings of data that can be utilised for further analysis. This unit defines the group by the mechanism used to group various data frames. It discusses the methods of creating groups and accessing specific data from the grouped data. Further, the data from the groups is often summarised to create dataframes of summaries, which can be useful in further analysis of operational data. For example, sales made by individuals within a company may be grouped to create monthly sales for the store or even yearly sales. This data may then be used to make a business plan for the next few years. This unit also discusses how dictionaries, series and functions can be used to create groups.

5.1 OBJECTIVES

After going through this unit, you should be able to:

- explain the purpose of the group by mechanism in data frames;
 - define groups on single or multiple columns and access the data of groups;
 - performing aggregate functions on grouped data;
 - dealing with row indexes while grouping;
 - perform grouping using a dictionary or a series data;
 - creating lambda functions to facilitate the grouping of data.
-

5.2 GROUP BY MECHANICS

In several data analysis tasks, you are required to generate statistics for a set of data that has similar data or is of the same category. This requires grouping of data on a data category. Thus, logically, data is divided into groups – one for each category and aggregate operations can be performed on those groups; thus, you can look into analysing the grouped data more effectively. Let us illustrate this with an example. Consider an organisation that has the following salary data of its employees:

Index #	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
1	Ravi M	Database	SQL	200000	2.5
2	Rehan D	Design	SQL	150000	1.5
3	Ben A	Database	Web Development	200000	2.0
4	Rai Y	Design	SQL	100000	1.0

Figure 1: Sample Data of Employees of an Organisation

Suppose you are interested in finding the average salary of employees in the two departments, 'Design' and 'Database'. To find this query, you may group the data records logically, viz. Index #0, 2, and 4 will form a subgroup of employees working in the 'Design' department, and Index #1 and 3 will comprise the second subgroup of employees in the 'Database' department. The data of these sub-groups can be analysed separately to find the average salary in each department. However, please note that you need not create these sub-groups physically; rather, a set of commands can be used to perform such a logical reorganisation of data.

We explain this grouping mechanism with the help of an example using Python. We use the data given in Figure 1 for this program. Figure 2 illustrates the program, written in Python, performing the grouping. The results of this program are shown in Figure 3.

```
import pandas as pd
#Part (a): Create the data frame employee and Print it
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                    'Rai Y'],
            'Department': ['Design', 'Database', 'Design', 'Database',
                           'Design'],
            'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
                               Development', 'SQL'],
            'Salary': [100000, 200000, 150000, 200000, 100000],
            'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
print("Part(a): Printing the Data frame of data of employees.",
      "\n")
print(Employee_data, "\n")
```

```
#Part (b): Create a Group on the Department and Display Group
by iterating over the groups
print('Part (b): Grouping the data using the attribute
      Department', "\n")
Employee_groups = Employee_data.groupby('Department')
print ('The grouped data object can be displayed by iterating
over the Groups:', "\n")
for employee_department, employee_data in Employee_groups:
    print("Department Name:", employee_department)
    print(employee_data)
    print("\n")
```

```
#Part (c): Performing operations on the groups of data
print ('Part (c): Performing Operations on Data of the Group:
Find the Mean salary of each Department and maximum of years of
working:')
print(Employee_groups['Salary'].mean(), "\n")
print(Employee_groups['YearsWorking'].max(), "\n")
print("To display data of one group: Use get_groups method")
print(Employee_groups.get_group('Design'), "\n")
```

```
#Part (d): Grouping data on multiple columns or attributes and
displaying the result by iterating over data. Also, performing
the operations on data grouped under multiple columns.
print('Part (d): Grouping the data on multiple columns or
attributes: Department and Specialisation', "\n")
Employee_multicol_groups = Employee_data.groupby(['Department',
'Specialisation'])
print('The grouped data on multiple columns can be displayed by
iterating over the Groups:', "\n")
for employee_department_specialisation, employee_data in
Employee_multicol_groups:
    print("Department Name and Specialisation:",
employee_department_specialisation)
    print(employee_data)
    print("\n")
print ('Performing Operations on Data of the Groups with
multiple columns: Find the Mean salary of each Department and
specialisations and maximum of years of working:')
print(Employee_multicol_groups['Salary'].mean(), "\n")
print(Employee_multicol_groups['YearsWorking'].max(), "\n")
```

Figure 2: Python Program for Grouping the Data on one or more Attributes

Please note the following points about Figure 2, given above:

1. Part (a) of the program creates the data frame for the data given in Figure 1. This data frame is named `Employee_data`. This data is printed in Part(a) of Figure 3.
2. Part(b) creates the logical grouping of data on the attribute/column 'Department', using `groupby()` function, which is applied to the data frame object `Employee_data`. Thus, the method is called as: `Employee_data.groupby('Department')`, creating a new object named `Employee_groups`.
You can display data from this `Employee_groups` object by iterating over the groups using the **for** loop. Please note that we have created two objects here, named `employee_department` and `employee_data`, which are displayed using separate print methods. The output of these print methods are shown in Part(b) of Figure 3. Please note that the Line showing the department name is displayed by the call to print method `print("Department Name:", employee_department)`, and the employee data of a department is displayed by the method `print(employee_data)`.
3. Part(c) demonstrates how aggregate functions like mean, max, etc. can be applied to the data groups. This part displays the mean or average Salary of all the employees of a group and the maximum number of years an employee has worked in that organisation. The result of these functions is shown in Part(c) of Figure 3. In addition, this part shows how you can print the data of one group only. The method used for this purpose is `get_group()`. The result of the use of this function is also shown in Part(c) of Figure 3.
4. Part(d) demonstrates the use of grouping data on multiple columns or attributes. This program creates an `Employee_multicol_groups` object that is grouped on two columns, 'Department' and 'Specialisation'. Please observe that the sequence of method calls to display the objects is the same, except now a group is defined on two columns. Accordingly, group records and group aggregates have changed.

Part(a): Printing the Data frame of data of employees.

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
1	Ravi M	Database	SQL	200000	2.5
2	Rehan D	Design	SQL	150000	1.5
3	Ben A	Database	Web Development	200000	2.0
4	Rai Y	Design	SQL	100000	1.0

Part (b): Grouping the data using the attribute Department

The grouped data object can be displayed by iterating over the Groups:

Department Name: Database

	Name	Department	Specialisation	Salary	YearsWorking
1	Ravi M	Database	SQL	200000	2.5
3	Ben A	Database	Web Development	200000	2.0

Department Name: Design

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
2	Rehan D	Design	SQL	150000	1.5
4	Rai Y	Design	SQL	100000	1.0

Part (c): Performing Operations on Data of the Group: Find the Mean salary of each Department and maximum of years of working:

Department

Database 200000.000000

Design 116666.666667

Name: Salary, dtype: float64

Department

Database 2.5

Design 3.5

Name: YearsWorking, dtype: float64

To display data of one group: Use get_groups method

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
2	Rehan D	Design	SQL	150000	1.5
4	Rai Y	Design	SQL	100000	1.0

Part (d): Grouping the data on multiple columns or attributes:

Department and Specialisation

The grouped data on multiple columns can be displayed by iterating over the Groups:

Department Name and Specialisation: ('Database', 'SQL')

	Name	Department	Specialisation	Salary	YearsWorking
1	Ravi M	Database	SQL	200000	2.5

Department Name and Specialisation: ('Database', 'Web Development')

	Name	Department	Specialisation	Salary	YearsWorking
3	Ben A	Database	Web Development	200000	2.0

Department Name and Specialisation: ('Design', 'SQL')

	Name	Department	Specialisation	Salary	YearsWorking
2	Rehan D	Design	SQL	150000	1.5
4	Rai Y	Design	SQL	100000	1.0

Department Name and Specialisation: ('Design', 'Web Development')

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5

Performing Operations on Data of the Groups with multiple columns:

Find the Mean salary of each Department and specialisations and maximum of years of working:

Department Specialisation

Database SQL 200000.0

Web Development 200000.0

Design SQL 125000.0

Web Development 100000.0

Name: Salary, dtype: float64

Department	Specialisation	
Database	SQL	2.5
	Web Development	2.0
Design	SQL	1.5
	Web Development	3.5

Name: YearsWorking, dtype: float64

Figure 3: Result of the Python Program for Grouping the Data

Let us list some of the key characteristics of the grouping mechanism.

1. One or more columns can be used to group the data; however, please note that these columns should be such that they contain some category of values.
2. You can apply group-based functions on the grouped data. These functions are discussed in a section later.
3. The groupby function of Python can create a new object that contains groups of data.
4. You can display data of one or all the groups.

Another interesting way of grouping data is by converting data columns to contain categorical datatype. For example, you can convert numerical age values to age categories by using a Python program or even a series of data to categorical. Why would you like to do so? The main reasons are:

1. Categorical data, even though it may consist of strings, is stored as a numerical value, which occupies very limited space. Thus, reducing the memory space required for storage, as well as processing time of data.
2. Further, it improves the performance of processing the data, as data is now of limited categories and integral in nature.
3. You can easily modify the labels of categories.
4. You can easily validate the data.

You may write Python programs for data type conversions.

5.2.1 Iterating Over Groups

Groups are created to perform a basic set of analytical functions on the data of the group. How will you be able to access data in each group or sub-group? One such method is to iterate over groups. This process can help in creating a separate list of records in an object, which can then be handled separately. Figure 2 part (b) shows an example of ways of handling groups by iterating over them. Please note that the *for* loop is performed on two objects: `employee_department` and `employee_data`. The first of the two objects contains the group name value, and the second object contains the records or rows that are part of the group. For example, in Figure 2, we have created grouping on 'Department' column. Since, there are only two values of 'Department' column, viz. 'Database' and 'Design', only two subgroups will be formed – one on 'Database' (with record numbers 1 and 3) and other on 'Design' (with record numbers 0, 2 and 4). So, the object `employee_department` will contain the sub-group values 'Database' and 'Design'; and `employee_data` will contain the records for 'Database' subgroup, i.e. record numbers 1 and 3; and the records of 'Design' subgroup, i.e. record numbers 0, 2 and 4. You may observe that Figure 3 part(b) displays these values.

In addition to displaying all the groups by iterating over the groups, you can also display data of only one subgroup. There are three basic methods of performing this operation. They are explained with the help of the program given in Figure 4 and its results given in Figure 5.

```
import pandas as pd
#Part (a) Create the data of employee, data frame and make
groups
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
#Create a Group on the Department
Employee_groups = Employee_data.groupby('Department')
```

```
#Part (b): Printing the subgroups: Method 1
print("You can use Three ways to process data of subgroups",
"\n")
print("Method 1: Showing data of 'Design' department by using
get_group() method:")
Employeees_in_Design = Employee_groups.get_group('Design')
print(Employeees_in_Design, "\n")
```

```
#Part (c): Printing the subgroups: Method 2
print("Method 2: Showing data of 'Design' Department by using
for and if statements:")
for employee_department, employee_data in Employee_groups:
    if employee_department == 'Design':
        print(employee_data)
        break # Stop after the desired group
print("\n")
```

```
#Part (d): Printing the subgroups: Method 3
print("Method 3: Filtering the grouped data by using lambda
function. Displaying only those groups where the Average Salary
> 150000")
filtered_department_group = Employee_groups.filter(lambda x:
x['Salary'].mean() > 150000)
print("Groups with Mean 'Salary' > 150000:")
print(filtered_department_group)
```

Figure 4: Program to process only one subgroup

Please note the following points about Figure 4:

1. Part (a) of Figure 4 creates the data frame and divides the data in groups on the column 'Department'. The name of the object that stores the groups is Employee_groups.
2. Part (b) has the code for displaying all records in the subgroup 'Design'. It uses the get_group() method. Please note the parameter used for this method. We have created a data frame named Employeees_in_Design by using the method get_group() with the parameter 'Design'. You can print this data frame to get the list of all the records of 'Design' subgroup. Please refer to the output of Method 1 in Figure 5.
3. Part (c) of Figure 4 demonstrates the use of for and if statements for iterating over the groups. The for statement iterates over the entire group, while the if statement is used to select the data of one or more groups of your choice. In this program segment, we have displayed the data of the employees of the 'Design' department. Please also note the use of the break statement, which terminates the for loop once the data for a particular group has been printed. Figure 5 Method 2 shows this output.
4. Part(d) of Figure 4 filters the data with the help of the lambda function of Python. The lambda function so created finds the mean of the 'Salary' of a

subgroup and displays the group whose average salary is more than 150000. Figure 5 Method 3 shows this output. You may please note that the average salary of 'Database' department employees is higher than 150000, and that is why the records of only that department are printed.

You can use Three ways to process data of subgroups

Method 1: Showing data of 'Design' department by using `get_group()` method:

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
2	Rehan D	Design	SQL	150000	1.5
4	Rai Y	Design	SQL	100000	1.0

Method 2: Showing data of 'Design' Department by using `for` and `if` statements:

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
2	Rehan D	Design	SQL	150000	1.5
4	Rai Y	Design	SQL	100000	1.0

Method 3: Filtering the grouped data by using `lambda` function. Displaying only those groups where the Average Salary > 150000
Groups with Mean 'Salary' > 150000:

	Name	Department	Specialisation	Salary	YearsWorking
1	Ravi M	Database	SQL	200000	2.5
3	Ben A	Database	Web Development	200000	2.0

Figure 5: Result of Program of Figure 4

Such grouping of data and dealing with subgroups is useful in performing separate operations on separate groups, as per the need.

5.2.2 Selecting a Columnar Subset of a Group

After grouping the data, you may like to perform separate group operations on the data. Further, you may not like to display some columns of the data frame at all. In addition, some of these columns can be used to compute the aggregated data. The program code of Figure 6 presents an example for the selection of columns.

```
import pandas as pd
#Part (a) Create the data of the employees
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
                  'Rai Y'],
           'Department': ['Design', 'Database', 'Design', 'Database',
                          'Design'],
           'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
                              Development', 'SQL'],
           'Salary': [100000, 200000, 150000, 200000, 100000],
           'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
#Create a Group on the Department
Employee_groups = Employee_data.groupby('Department')
```

```
#Part (b): Selecting only the Salary and YearsWorking columns
of data along with the grouped column.
selected_columns = Employee_groups[['Salary', 'YearsWorking']]
print("Part (b): Selecting only Salary and YearsWorking columns
of data along with the grouped column.")
for employee_department, employee_data in selected_columns:
    print(employee_department)
    print(employee_data)
```



```
#Part (c): Showing the Grouped Columns using the Aggregate
functions on Salary and YearsWorking data.
print("\nPart (c): Showing the Grouped Columns using the
Aggregate functions on Salary and YearsWorking data..")
AggregateOfData = selected_columns.aggregate(
    Department_Salary=('Salary', 'sum'),
    Max_Experience=('YearsWorking', 'max'),
    Min_Experience=('YearsWorking', 'min')
).reset_index()
print("The Aggregated Data:")
print(AggregateOfData)
```

Figure 6: Selecting a Column Subset and Creating Aggregate Data

The program of Figure 6 is explained as under:

1. Part (a) of this program creates the data, data frame and groups the data using the column 'Department'
2. Part (b) of Figure selects only two columns from the Employee_group created in Part (a). These two columns are 'Salary' and 'YearsWorking'. Please note this is communicated by enclosing the column names as a list or in other words, are put in square brackets ([]). You may please note that Part(b) of Figure 7 shows this output. You can clearly observe that the data output contains the department of the group and the two columns 'Salary' and YearsWorking'.
3. Part(c) of this program demonstrates the use of aggregate function of selected columns. Please note that a new aggregate function definition is specified in this part. It creates a new column named Department_Salary, which finds the sum of the 'Salary' of all the employees in a group. Similarly, the Max_Experience and Min_Experience columns are created from the column 'YearsWorking' by using functions max and min, respectively. Finally, the index is reset by using reset_index() method call, thus, reindexing the new set of records. Finally, the output data frame AggregateOfData is printed using print method. Output of this program is shown in Part(c) of Figure 7. Please observe that there are only two records in the output as there are only two subgroups in the data, viz. 'Database' and 'Design'. You can find the salary of every employee in Part (b) of output and check whether the Department_Salary is same. They should be the same.

```
Part (b): Selecting only Salary and YearsWorking columns of data along
with the grouped column.
Database
   Salary  YearsWorking
1  200000             2.5
3  200000             2.0
Design
   Salary  YearsWorking
0  100000             3.5
2  150000             1.5
4  100000             1.0
```

```
Part (c): Showing the Grouped Columns using the Aggregate functions on
Salary and YearsWorking data..
The Aggregated Data:
   Department  Department_Salary  Max_Experience  Min_Experience
0    Database             400000             2.5             2.0
1     Design             350000             3.5             1.0
```

Figure 7: Output of Selection of a Column Subset and Creating Aggregate Data

In this section, we have shown an example of data aggregation. Let us discuss about data aggregation methods used in Python in the next section.

Check Your Progress 1

A Business organisation collects the following data of its sales of different categories of products in different years by various regional offices:

Index#	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
0	Grocery	North	500	2023-12-31
1	Medicine	South	490	2024-12-31
2	Grocery	South	700	2024-12-31
3	Educational	North	1000	2024-12-31
4	Educational	North	600	2023-12-31
5	Medicine	South	900	2023-12-31

Table 1: Data for check your progress

Create a data frame of the data given above and perform the following tasks on the data:

1. Rearrange the data in groups using the Category of product and display these subgroups.

.....
.....
.....
.....

2. Rearrange the data to display the total sales of the 'Educational' Category only.

.....
.....
.....
.....

3. Rearrange the data to show Region-wise total sales of each category.

.....
.....
.....
.....

4. Show records of only those regions whose total sales are 2100 thousand or more.

.....
.....
.....
.....

5.3 DATA AGGREGATION

In the context of data wrangling, data aggregation primarily involves the process of summarising data. Such summarisation of data is possible only if the data is well organised. Thus, raw data is first processed and combined to create structured data. For example, a University may keep information about students' assignment results in several files. This data may be processed to create consistent data about the student's grade cards. This data can then be summarised to create information about the number of students who passed in a University programme. The following table displays the list of the most useful aggregate functions:

Method	Purpose
sum()	Finds the sum of the parameter values
count()	Counts the number of occurrences of a parameter value. These values will not be counted if the value is Null.
mean()	Finds the mean or average of a set of values
min()	Finds the minimum from a set of values
max()	Finds the maximum from a set of values
std()	Finds the standard deviation of a set of data.

Figure 8: Functions/Methods of Data Aggregation

You can apply several aggregation functions to a column. This is explained in the next sub-section.

5.3.1 Multiple Aggregate Functions in a Column

Aggregate functions provide a summarised view of data. In certain situations, you want to apply several aggregate functions on a column. This enhances the efficiency of data processing. For example, if you want to find the average salary, total salary and maximum in a department, you will be required to apply multiple aggregate functions in a single aggregation operation. Figure 9, Part (a) applies multiple aggregate functions, viz. 'count', 'mean' and 'std' on the column named 'YearsWorking'. You may observe the output given in Figure 10, Part (a), which clearly shows the separate values of each aggregated output. Please also note that the overall aggregation of data is for every department, and the values of count, mean, and standard deviation are calculated for each department's data in the column 'YearsWorking'.

```
import pandas as pd
#Create the data of the employees
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A', 'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
```

```
#Part (a): Performing Multiple Functions on YearsWorking column.
print("Part (a): Multiple Functions on YearsWorking column:")
Employee_groups =
Employee_data.groupby('Department')['YearsWorking'].aggregate(['count'
, 'mean', 'std'])
print(Employee_groups)
```

```
#Part (b): Performing Multiple Functions on different columns
print("\nPart (b): Multiple Functions on Different columns:")
Employee_groups2 = Employee_data.groupby('Department').aggregate({
'YearsWorking': ['count', 'mean'],
'Salary': ['sum', 'max']})
print(Employee_groups2)
```

```
#Part (c): Row Indexes in Aggregated Data
print("\nPart (c): Row indexes in Aggregated Data:")
Employee_groups3 = Employee_data.groupby('Department',
as_index=False)['YearsWorking'].aggregate(['count', 'mean', 'std'])
print(Employee_groups3)
```

Figure 9: Multiple Aggregations on the Columns

```
Part (a): Multiple Functions on YearsWorking column:
      count  mean      std
Department
Database      2  2.25  0.353553
Design        3  2.00  1.322876
```

Part (b): Multiple Functions on Different columns:				
YearsWorking	Salary			
	count	mean	sum	max
Department				
Database	2	2.25	400000	200000
Design	3	2.00	350000	150000

Part (c): Row indexes in Aggregated Data:				
	Department	count	mean	std
0	Database	2	2.25	0.353553
1	Design	3	2.00	1.322876

Figure 10: Result of Multiple Aggregations on the Columns

In Part(b) of Figure 9, different multiple aggregate functions are applied to different columns. You may observe that the aggregate functions ‘count’ and ‘mean’ are applied to the ‘YearsWorking’ column, and ‘sum’ and ‘max’ are applied to the ‘Salary’ column. The results of this are shown in Part (b) of Figure 10. Please also note that the basic grouping is on the Department column.

Row Indexes in Aggregated Data

When you display the aggregated data on some functions, the column on which you group the data becomes the index of the resultant aggregated data. For example, in Part (a) of Figure 9, grouping is performed on the ‘Department’ column, thus, an index is created by default on this variable. Therefore, when you display the content of the aggregated data, it is indexed on Department. When you print the aggregated data, this default ‘Department’ index is used to display the result (refer to Figure 10 Part (a)). If you do not want your results using this default index, then you use the option *as_index=False* in the *groupby* clause. The program in Figure 9 Part (c) makes the row indexes as default index. You may observe the result of the code of Part (c) of Figure 9 in Part (c) of Figure 10. The row numbers are clearly visible in the output.

The basic advantages of using these row indexes are that the data frame of the output is in standard form and can be exported easily to other formats.

5.3.2 Column-wise Aggregation

Interestingly, Python also allows you to perform separate aggregations for separate columns of the data. One such example of this feature is shown in Part (c) of Program 6, where different aggregate functions have been selected on a group by object selected *columns* using the *aggregate* method call. You may notice that the ‘Salary’ column is aggregated using the *sum* method, whereas the ‘YearsWorking’ column is aggregated using the *max* and *min* methods.

Another example of aggregation is presented in Figure 11.

```
import pandas as pd
#Part (a) Create the data of the employees
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
' Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
#Part (a): Create Groups on the Department and Specialisation
and print them by iterating over groups
print("Part (a): The Groups on Department and Specialisation:
\n")
```

```
Employee_Multi_group =
Employee_data.groupby(['Department', 'Specialisation'])
for employee_depart_Spec, employee_data_multi in
Employee_Multi_group:
    print(employee_depart_Spec)
    print(employee_data_multi, "\n")

#Part (b) Aggregating Data using different methods
AggregateOfData = Employee_Multi_group.aggregate(
NoOfEmployees = ('Name', 'count'),
Total_Salary=('Salary', 'sum'),
Mean_Exp=('YearsWorking', 'mean')
).reset_index()
print("\nPart (b) Aggregated Data using different methods:")
print(AggregateOfData)
```

Figure 11: Aggregating Data

The program given in Figure 11 is explained as under:

1. Figure 11 Part (a) creates the data frame Employee_data, which contains the hypothetical data of the employees. This data frame is then grouped on multiple columns, namely 'Department' and 'Specialisation'. This grouped data is printed by iterating over the groups. You can observe the output of this grouped data in Figure 12, part(a). The output first displays the value of the Department and Specialisation at the top, followed by all the data frames of that subgroup.
2. Part (b) of the program demonstrates how different types of aggregation methods can be applied to different columns. In this case, the column 'Name' is being counted, column 'Salary' is being summed, and column 'YearsWorking' is being averaged. Please observe the output of this part of the program in Figure 12, part (b). You can verify the results after looking into the data shown in Part (a) of Figure 12. For example, in Figure 12 part (a), the ('Design', 'SQL') subgroup contains two rows (or employees), the sum of their salaries would be 15000+10000=25000, and the mean experience is $(1.5+1.0)/2 = 1.25$. Please compare these computed values to the index 2 row of Part (b) of Figure 12.

Part (a): The Groups on Department and Specialisation:

```
('Database', 'SQL')
  Name Department Specialisation Salary YearsWorking
1  Ravi M   Database           SQL  200000         2.5

('Database', 'Web Development')
  Name Department Specialisation Salary YearsWorking
3  Ben A   Database Web Development  200000         2.0

('Design', 'SQL')
  Name Department Specialisation Salary YearsWorking
2  Rehan D   Design           SQL  150000         1.5
4   Rai Y   Design           SQL  100000         1.0

('Design', 'Web Development')
  Name Department Specialisation Salary YearsWorking
0  Arav S   Design Web Development  100000         3.5
```

Part (b) Aggregated Data using different methods:

	Department	Specialisation	NoOfEmployees	Total_Salary	Mean_Exp
0	Database	SQL	1	200000	2.50
1	Database	Web Development	1	200000	2.00
2	Design	SQL	2	250000	1.25
3	Design	Web Development	1	100000	3.50

Figure 12: Result of the Program in Figure 11 on Aggregating Data

Create a data frame of the data given in Table 1 of Check Your Progress 1 and perform the following tasks on the data:

1. Find the minimum and maximum sales for each category. Also, find the number of regions where products of this category are sold.

.....

2. Aggregate the Sales data category-wise and create a Row index for this data.

.....

3. Create aggregated data on sales and count on the number of years of data for the two columns - Region and Category.

.....

5.4 GROUPING WITH DICTIONARIES AND SERIES

In the previous sections, we have discussed about grouping a dataframe using one or multiple columns. In this section, we will discuss how grouping can be done with the help of a dictionary or a series. When grouping by dictionary, you create a mapping of a different set of values or column names, whereas a series is used to assign certain labels on which grouping can be performed in rows. Grouping is performed using a dictionary or a series when no column is available for grouping or when grouping is needed on metadata.

```
import pandas as pd
#Part (a) Create the data of the employees, which is indexed by
the employee name
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'YearsWorking': [3.5, 2.5, 1.5, 2.0, 1.0]}
#Part (a.1): Creating a data frame of the data of employees
Employee_data = pd.DataFrame(empdata)
Indexed_Emp_data = Employee_data.set_index('Name').sort_index()
print("Part (a.1): Employee Data Frame indexed on Name and
sorted:\n", Indexed_Emp_data)
```

```
#Part (a.2) Create Project mapping using a Dictionary
assigned_project = {'Arav S': 'e-commerce', 'BenA': 'e-commerce',
'Rai Y': 'M-commerce', 'Ravi M': 'M-commerce', 'Rehan D': 'e-
commerce'}
print("\nPart (a.2): Dictionary used for
Grouping:\n", assigned_project)
```

```
# Part (a.3): Performing GroupBy of a Dataframe using a
Dictionary. Please note the use of 'axis=0', as this creates a
grouping by row indexes. For column grouping, use 'axis=1'.
grouped_by_projects =
Indexed_Emp_data.groupby(assigned_project, axis=0).aggregate({
    'YearsWorking': ['count', 'mean'],
    'Salary': ['sum', 'max']})
#Please note that aggregation is also performed using a
dictionary.
print("\nPart (a.3): Grouped by Projects by using dictionary on
assigned_Projects:\n", grouped_by_projects)
```

```
#Part (b)Group by using a Series
#Part (b.1) Creating a series
Experience_series = pd.Series(['Two or More Years', 'Two or
More Years', 'Less than two Years', 'Two or More Years', 'Less
than two Years'])
print("\nPart (b.1): The Grouping Series:\n",
Experience_series)
```

```
# Part (b.2): Grouping the employee data by the experience
Series and calculating the sum of Salary in that group.
grouped_by_experience =
Employee_data.groupby(Experience_series).aggregate({
    'YearsWorking': ['count', 'mean'],
    'Salary': ['sum']})
print("\nPart (b.2): Employee data frame:\n", Employee_data)
print("\nPart (b.2): Grouping of Employee data frame on
experience using a series:\n", grouped_by_experience)
```

Figure 13: Grouping with Dictionary and Series

Please observe the following points about Figures 13 and 14.

1. Part (a.1) of Figure 13 contains the commands of creation a dataframe of employee's data. Please note that in this part, an indexed data frame on column 'Name' has been created. Here, we are assuming that employee names are unique. The result of these commands is displayed in Part (a.1) of Figure 14. Please observe that the index of this output is Name, and this index is sorted, creating a sorted data frame on employee name.
2. In Part (a.2) of Figure 13, a dictionary, which maps an employee to the Project on which s/he is working, has been created. This dictionary is printed in Part (a.2) of Figure 14.
3. Part (a.3) of Figure 13, creates group using groupby clause on the data frame Indexed_Emp_data. Please observe that the grouping has been performed by using the dictionary given in Part(a.2). Please also observe the use of 'axis=0', which makes groups of the rows. In case you use 'axis=1', then the groups would be created for the columns. The grouping is followed by the aggregation functions, which are also presented with the help of dictionary. The output of this part of the program is shown in Part (a.3) of Figure 14. You may check the results.
4. Part (b) of Figure 13 shows a program that uses a series to group the employee data frame. A categorical series that consists of only two values, viz., experience 'Two or More Years' or 'Less than two Years' for various rows of the employee data frame. Par (b.1) of Figure 14 shows this data series.
5. In part (b.2) of Figure 13, the Employee_data data frame is grouped by using the Experience Series. The dictionary of aggregate functions. The output of the Employee_data dataframe and employee data after the grouping and application of the aggregate function is shown in Figure 14 Part (b.2).

Part (a.1): Employee Data Frame indexed on Name and sorted:

Name	Department	Specialisation	Salary	YearsWorking
Arav S	Design	Web Development	100000	3.5
Ben A	Database	Web Development	200000	2.0
Rai Y	Design	SQL	100000	1.0
Ravi M	Database	SQL	200000	2.5
Rehan D	Design	SQL	150000	1.5

Part (a.2): Dictionary used for Grouping:

```
{'Arav S': 'e-commerce', 'BenA': 'e-commerce', 'Rai Y': 'M-commerce', 'Ravi M': 'M-commerce', 'Rehan D': 'e-commerce'}
```

Part (a.3): Grouped by Projects by using dictionary on assigned Projects:

Name	YearsWorking	Salary			
		count	mean	sum	max
M-commerce		2	1.75	300000	200000
e-commerce		2	2.50	250000	150000

Part (b.1): The Grouping Series:

```
0    Two or More Years
1    Two or More Years
2    Less than two Years
3    Two or More Years
4    Less than two Years
dtype: object
```

Part (b.2): Employee data frame:

	Name	Department	Specialisation	Salary	YearsWorking
0	Arav S	Design	Web Development	100000	3.5
1	Ravi M	Database	SQL	200000	2.5
2	Rehan D	Design	SQL	150000	1.5
3	Ben A	Database	Web Development	200000	2.0
4	Rai Y	Design	SQL	100000	1.0

Part (b.2): Grouping of Employee data frame on experience using a series:

YearsWorking	Salary		
	count	mean	sum
Less than two Years	2	1.250000	250000
Two or More Years	3	2.666667	500000

Figure 14: Output of Program 13, Grouping with Dictionary and Series

In the next section, we discuss about the grouping of data using functions.

5.5 GROUPING WITH FUNCTIONS

Functions are, in general, useful for grouping data on date/time columns or a part of a string. Further, the functions can be used to group data frames by index or by column. In this section, the grouping by function has been explained with the help of an example involving grouping by date and the first character of the name. For this example, we have changed one of the columns of the data, i.e., instead of years of working, we have created a column named DateOfJoining, which contains the data of date type. Since Python does not have a built-in date type, we use the datetime module of Python for dealing with dates. Please refer to line 2 of Figure 15, which imports the datetime

module. Figure 15 shows a program for grouping by functions, and Figure 16 shows the results of this program.

```
import pandas as pd
from datetime import datetime

#Part (a) Create the data of the employees, which is indexed by
the employee name
empdata = {'Name': ['Arav S', 'Ravi M', 'Rehan D', 'Ben A',
'Rai Y'],
'Department': ['Design', 'Database', 'Design', 'Database',
'Design'],
'Specialisation': ['Web Development', 'SQL', 'SQL', 'Web
Development', 'SQL'],
'Salary': [100000, 200000, 150000, 200000, 100000],
'DateOfJoining': [datetime(2023, 2, 16), datetime(2024, 12, 2),
datetime(2024, 10, 23), datetime(2025, 4, 13), datetime(2023,
8, 31)]}
#Creating a data frame of the data of employees and print it
Employee_data = pd.DataFrame(empdata)
print("Part (a): Employee Data Frame i:\n", Employee_data)

# Part (b): Creating a Group on the First character of the
'Name' by using a Lambda function
GroupbyFirstCharacterofName = Employee_data.groupby(lambda x:
Employee_data.loc[x, "Name"][0])
# Displaying the group using an iterator
print("\nPart (b): Grouping using First Character of the Name")
for Name_first_char, employees in GroupbyFirstCharacterofName:
    print("\nEmployee with Name starting with",
Name_first_char,":")
    print(employees)

# Part (c): Creating a Group on DateOfJoining column 'using a
Lambda function and performing aggregate functions
yearly_joined = Employee_data.groupby(
    lambda x: Employee_data.loc[x,
'DateOfJoining'].strftime("%Y")
).aggregate({
    'Name': ['count'],
    'Salary': ['mean']})
print("\nPart (c): Grouping by using Year")
print("\nAnnual employees joining count with Mean Salary",
yearly_joined)
```

Figure 15: Grouping with Functions

The details of the Figure 15 and Figure 16 are explained below:

1. In part (a) of Figure 15, first we have imported *pandas* library. This is followed by import of *datetime* module. Next, a data frame of employee's data is created and printed. Please note that the column *DateOfJoining* contains the date of joining of employees. Please also note the use of *datetime()* method to convert the parameters as a date. Please note the parameters are year, month and day. Figure 16 part (a) shows the output of the frame. Please observe the *DateOfJoining* column in this output.
2. Part (b) of Figure 15 contains the code `Employee_data.groupby(lambda x: Employee_data.loc[x, "Name"][0])`, which groups the *Employee_data* data frame using the rule specified by the stated lambda function. Please note that a lambda function is an anonymous function of Python, which iterates over all the rows to return the

first character of the 'Name' column of each row. How? The variable `x` defines the iteration over `rowindex` value of the data frame `Employee_data` over the column "Name", and `[0]` refers to the first character of the Name column of each row index value. Please note that `.loc` selects data from the dataframe based on the index value specified by `x`. Thus, this lambda function will return the first character of the 'Name' column of every row, and that will be used for grouping. The output of this grouping is displayed by iterating over the groups. Figure 16 Part (b) shows the grouped output.

- Part (c) of Figure 16 creates the grouping of data using the Year of the `DateOfJoining` column. Please notice the `Employee_data.groupby(lambda x: Employee_data.loc[x, "DateOfJoining"].strftime("%Y"))...` groups the dataframe `Employee_data` using the lambda function. This function selects the year column of every row, as specified by `strftime("%Y")` method of `datetime` module. Please note the full form of *strftime* is string format time and parameter `("%Y")` selects only the year data from the `DateOfJoining` column, which is used to group the rows of the dataframe. In this part, we have further performed the aggregation of data in the group, so as to highlight the importance of grouping. You may observe the output in Part (c) of Figure 16.

Thus, lambda function based grouping is useful for creating data when you need to perform grouping based on some computations.

Part (a): Employee Data Frame i:

	Name	Department	Specialisation	Salary	DateOfJoining
0	Arav S	Design	Web Development	100000	2023-02-16
1	Ravi M	Database	SQL	200000	2024-12-02
2	Rehan D	Design	SQL	150000	2024-10-23
3	Ben A	Database	Web Development	200000	2025-04-13
4	Rai Y	Design	SQL	100000	2023-08-31

Part(b): Grouping using First Character of the Name

Employee with Name starting with A :

	Name	Department	Specialisation	Salary	DateOfJoining
0	Arav S	Design	Web Development	100000	2023-02-16

Employee with Name starting with B :

	Name	Department	Specialisation	Salary	DateOfJoining
3	Ben A	Database	Web Development	200000	2025-04-13

Employee with Name starting with R :

	Name	Department	Specialisation	Salary	DateOfJoining
1	Ravi M	Database	SQL	200000	2024-12-02
2	Rehan D	Design	SQL	150000	2024-10-23
4	Rai Y	Design	SQL	100000	2023-08-31

Part (c): Grouping by using Year

Annual employees joining count with Mean Salary

	Name	Salary
	count	mean
2023	2	100000.0
2024	2	175000.0
2025	1	200000.0

Figure 16: Output of Figure 15 of Grouping with Functions

Check Your Progress 3

Create a data frame of the data given in Table 1 of Check Your Progress 1 and perform the following tasks on the data:

1. Assuming that all sales are done by the stores, create a mapping of stores as a dictionary, as per the data given below, and use this dictionary to calculate store-wise total sales along with the number of Stores.

Mapping: Store = {0: 'ABC Store', 1: 'ABC Store', 2: 'XYZ Store', 3: 'XYZ Store', 4: 'ABC Store', 5: 'ABC Store'}

.....
.....
.....

2. Assuming the following series of feedback received for every row of sales, create a grouping to find the performance of sales based on this series.
['Good', 'Excellent', 'Good', 'Excellent', 'Good', 'Excellent']

.....
.....
.....

3. Using a lambda function, find the total sales of each year.

.....
.....
.....

5.6 SUMMARY

This unit discusses the groupby mechanism using examples of Python programming. The following topics have been covered in the unit:

- Creating a group on one or multiple columns.
- Iterating the data of the groups using *for* loop.
- Selecting only a subgroup from the entire data, like columnar subset.
- Performing data aggregation over the groups to create summarised data frames.
- Creating row indexes in the aggregated data.
- Performing different aggregations on different columns.
- Creating groups using dictionaries and series.
- Using lambda functions to perform complex grouping.

5.7 ANSWERS/SOLUTIONS

Check Your Progress 1:

```
import pandas as pd
from datetime import datetime
#Part (0) Create the data of Sales
salesdata = {'CategoryOfProduct': ['Grocery',
'Medicine', 'Grocery', 'Educational', 'Educational',
'Medicine'], 'SalesRegion': ['North', 'South', 'South', 'North',
'North', 'South'], 'SalesinThousands': [500, 490, 700, 1000, 600,
900], 'YearEndingOn': [datetime(2023, 12, 31), datetime(2024,
12, 31), datetime(2024, 12, 31), datetime(2024, 12, 31),
datetime(2023, 12, 31), datetime(2023, 12, 31)]}
```

```
#Creating a data frame of the salesdata
yearlysales = pd.DataFrame(salesdata)
print(yearlysales)
```

```
#Ans 1: Group by Category and display of subgroups
print('\nAns 1: Grouping the data using the attribute Category
of Products:')
Catagory_groups = yearlysales.groupby('CategoryOfProduct')
print ('The grouped data object can be displayed by iterating
over the Groups:', "\n")
for category, yearly_data in Catagory_groups:
    print("Category of Product:", category)
    print(yearly_data)
```

```
#Ans 2: Rearrange the data to display 'Educational' category
and sum the sales
print ('\nAns 2: Displaying the total Sales of Educational
Category:')
Educational_Category_sales =
Catagory_groups.get_group('Educational')
print("\nTotal Sales of Educational category:",
Educational_Category_sales['SalesinThousands'].sum())
```

```
#Ans 3: Show Region-wise total sales of each category
print('\nAns 3: Region-wise total sales of each category', "\n")
Region_category_groups = yearlysales.groupby(['SalesRegion',
'CategoryOfProduct'])
print(Region_category_groups['SalesinThousands'].sum(), "\n")
```

```
#Ans 4: Show records of only those regions whose total sales
are 2100 thousand or more.
Region_groups = yearlysales.groupby(['SalesRegion'])
print("Ans 4: Showing the Region wise Total Sales\n",
Region_groups['SalesinThousands'].sum())
filtered_regional_group = Region_groups.filter(lambda x:
x['SalesinThousands'].sum() >= 2100)
print("\nThe Records of Group fulfilling the condition of
filtered_regional_group\n", filtered_regional_group)
```

	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
0	Grocery	North	500	2023-12-31
1	Medicine	South	490	2024-12-31
2	Grocery	South	700	2024-12-31
3	Educational	North	1000	2024-12-31
4	Educational	North	600	2023-12-31
5	Medicine	South	900	2023-12-31

```
Ans 1: Grouping the data using the attribute Category of Products:
The grouped data object can be displayed by iterating over the Groups:
```

```
Category of Product: Educational
CategoryOfProduct SalesRegion SalesinThousands YearEndingOn
3 Educational North 1000 2024-12-31
4 Educational North 600 2023-12-31
```

Category of Product: Grocery				
	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
0	Grocery	North	500	2023-12-31
2	Grocery	South	700	2024-12-31
Category of Product: Medicine				
	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
1	Medicine	South	490	2024-12-31
5	Medicine	South	900	2023-12-31

Ans 2: Displaying the total Sales of Educational Category:

Total Sales of Educational category: 1600

Ans 3: Region-wise total sales of each category

SalesRegion	CategoryOfProduct	
North	Educational	1600
	Grocery	500
South	Grocery	700
	Medicine	1390

Name: SalesinThousands, dtype: int64

Ans 4: Showing the Region wise Total Sales

SalesRegion	
North	2100
South	2090

Name: SalesinThousands, dtype: int64

The Records of Group fulfilling the condition of
filtered_regional_group

	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
0	Grocery	North	500	2023-12-31
3	Educational	North	1000	2024-12-31
4	Educational	North	600	2023-12-31

Check Your Progress 2

```
import pandas as pd
from datetime import datetime
#Part (0) Create the data of Sales
salesdata = {'CategoryOfProduct': ['Grocery',
'Medicine', 'Grocery', 'Educational', 'Educational',
'Medicine'], 'SalesRegion': ['North', 'South', 'South', 'North',
'North', 'South'], 'SalesinThousands': [500, 490, 700, 1000, 600,
900], 'YearEndingOn': [datetime(2023, 12, 31), datetime(2024,
12, 31), datetime(2024, 12, 31), datetime(2024, 12, 31),
datetime(2023, 12, 31), datetime(2023, 12, 31)]}
#Creating a data frame of the salesdata
yearlysales = pd.DataFrame(salesdata)
print(yearlysales)
```

#Ans 1: Multiple Aggregate Functions on different columns: 1.
Find the minimum and maximum sales for each category. Also,
find the number of regions where products of this category are
sold.

```
print("\nAns 1: Category wise Minimum and Maximumon and number
of regions where it is sold:")
Category_groups =
yearlysales.groupby('CategoryOfProduct').aggregate({
    'SalesinThousands': ['min', 'max'],
    'SalesRegion': ['count']})
print(Category_groups)
```

```
#Ans 2: Aggregate the Sales data category-wise and create a Row
index for this data.
print("\nAns 2: Row indexes in Aggregated Data:")
Category_groups_indexed =
yearlysales.groupby('CategoryOfProduct',
as_index=False)['SalesinThousands'].aggregate(['min', 'max',
'sum'])
print(Category_groups_indexed)
```

```
#Ans 3: Create the aggregated data on sales and count on years
of data for the two columns, Region and Category.
print("\nAns 3: Create the aggregated data for Region and
Category:")
Region_Multi_group = yearlysales.groupby(['SalesRegion',
'CategoryOfProduct'])
AggregateOfData = Region_Multi_group.aggregate(
CountOfYears = ('YearEndingOn', 'count'),
Total_Sales=('SalesinThousands', 'sum') ).reset_index()
print("Aggregated Data:")
print(AggregateOfData)
```

	CategoryOfProduct	SalesRegion	SalesinThousands	YearEndingOn
0	Grocery	North	500	2023-12-31
1	Medicine	South	490	2024-12-31
2	Grocery	South	700	2024-12-31
3	Educational	North	1000	2024-12-31
4	Educational	North	600	2023-12-31
5	Medicine	South	900	2023-12-31

Ans 1: Category wise Minimum and Maximum on and number of regions where it is sold:

CategoryOfProduct	SalesinThousands		SalesRegion
	min	max	count
Educational	600	1000	2
Grocery	500	700	2
Medicine	490	900	2

Ans 2: Row indexes in Aggregated Data:

	CategoryOfProduct	min	max	sum
0	Educational	600	1000	1600
1	Grocery	500	700	1200
2	Medicine	490	900	1390

Ans 3: Create the aggregated data for Region and Category:
Aggregated Data:

	SalesRegion	CategoryOfProduct	CountOfYears	Total_Sales
0	North	Educational	2	1600
1	North	Grocery	1	500
2	South	Grocery	1	700
3	South	Medicine	2	1390

Check Your Progress 3

```
import pandas as pd
from datetime import datetime
#Create the data of Sales
salesdata = {'CategoryOfProduct': ['Grocery',
```

```
'Medicine', 'Grocery', 'Educational', 'Educational',
'Medicine'], 'SalesRegion': ['North', 'South', 'South', 'North',
'North', 'South'], 'SalesinThousands': [500, 490, 700, 1000, 600,
900], 'YearEndingOn': [datetime(2023, 12, 31), datetime(2024,
12, 31), datetime(2024, 12, 31), datetime(2024, 12, 31),
datetime(2023, 12, 31), datetime(2023, 12, 31)]]}
#Creating a data frame of the salesdata
yearlysales = pd.DataFrame(salesdata)
```

```
#Ans 1: Create a Store Mapping using a Dictionary and Perform
Grouping using this dictionary.
Store = {0: 'ABC Store', 1: 'ABC Store', 2: 'XYZ Store', 3: 'XYZ
Store', 4: 'ABC Store', 5: 'ABC Store'}
#Please note the use of 'axis=0' for grouping by row indexes.
grouped_by_Store = yearlysales.groupby(Store,
axis=0).aggregate({'SalesinThousands': ['count', 'sum']})
print("\nAns 1: Grouping by dictionary on Store:\n",
grouped_by_Store)
```

```
#Ans 2: Create a Series and Perform Grouping using the series.
CustomerExpereince = pd.Series(['Good', 'Excellent', 'Good',
'Excellent', 'Good', 'Excellent'])
#Please note the use of 'axis=0' for grouping by row indexes.
grouped_by_CustExp =
yearlysales.groupby(CustomerExpereince).aggregate({'SalesinThou
sands': ['count', 'sum']})
print("\nAns 2: Grouping by Series on Customer Experience:\n",
grouped_by_CustExp)
```

```
# Ans 3: Creating a Group on YearEndingOn column using a Lambda
function and performing aggregate functions
yearly_sales = yearlysales.groupby(
    lambda x: yearlysales.loc[x, "YearEndingOn"].strftime("%Y")
).aggregate({'SalesinThousands': ['sum']})
print("\nAns 3: Grouping Sales by using Year")
print("\nAnnual Total of Sales\n", yearly_sales)
```

```
Ans 1: Grouping by dictionary on Store:
      SalesinThousands
           count  sum
ABC Store         4 2490
XYZ Store         2 1700
```

```
Ans 2: Grouping by Series on Customer Experience:
      SalesinThousands
           count  sum
Excellent         3 2390
Good              3 1800
```

```
Ans 3: Grouping Sales by using Year

Annual Total of Sales
      SalesinThousands
           sum
2023         2000
2024         2190
```